

PAC — Plataforma de Assembleias Condominiais

Documento de Arquitetura e Roteiro de Apresentação

Equipe APEX

Arthur Magno • João Gabriel • Maria Eduarda Gomes

Baseado na ERSw PAC versão 1.1 • Stack: Python 3 + Flask + SQLite

Teresina – PI, Julho de 2026

1. Visão geral

O **PAC** é um sistema web *mobile-first* para gestão de assembleias e votações de condomínio (ERSw 1.1), com dois perfis: o **Administrador/Síndico**, que configura, inicia e encerra votações e consulta resultados; e o **Proprietário**, vinculado a zero ou vários lotes, que vota e consulta resultados. Este projeto implementa **exatamente quatro funcionalidades**, rastreáveis à ERSw:

#	FUNCIONALIDADE	CASO DE USO	REQUISITOS	REGRAS
1	Entrar no sistema	UC1 — Autenticação	RF9, RNF4	—
2	Criar votação	UC4 — Configuração de Votações	RF5	RN3, RN7, RN8, RN9
3	Executar votação	UC5 — Execução de Votação	RF6	RN1–RN5, RN8
4	Verificar o resultado	UC6 — Apuração e Resultado	RF7	RN5, RN6

Fora do escopo: gestão de usuários/lotes (UC2), reuniões/pautas/anexos (UC3) e PDF do resultado (RF8) — supridos por um **script de seed** que popula o banco com os cenários de teste (inclusive Maria com 2 lotes = peso 2,40 e um proprietário sem lote, para demonstrar RN1/RN2).

2. Arquitetura escolhida e justificativa

Adotamos **arquitetura em camadas** (espírito MVC), com três camadas bem separadas:

- **Apresentação** — telas Jinja2 + rotas Flask. As rotas apenas traduzem HTTP em chamadas de caso de uso. **Nenhuma regra de negócio vive aqui.**
- **Serviço / Casos de uso** — o coração do sistema: um pacote por caso de uso da ERSw (`uc1_autenticacao`, `uc4_criar_votacao`, `uc5_executar_votacao`, `uc6_resultado`), arquivos pequenos, um por responsabilidade. **Toda regra RN1–RN9 mora aqui**, com comentário-cabeçalho citando a regra.
- **Persistência** — um modelo por entidade da ERSw sobre SQLite; RN4 e RN5 são reforçadas **também por constraints e trigger no banco.**

Por que camadas? O critério dominante é didático: na apresentação, cada requisito é apontado direto no arquivo que o implementa. "Onde mora a RN4?" tem resposta única:
`casos_de_uso/uc5_executar_votacao/votar.py + constraint UNIQUE em config/banco.py`.

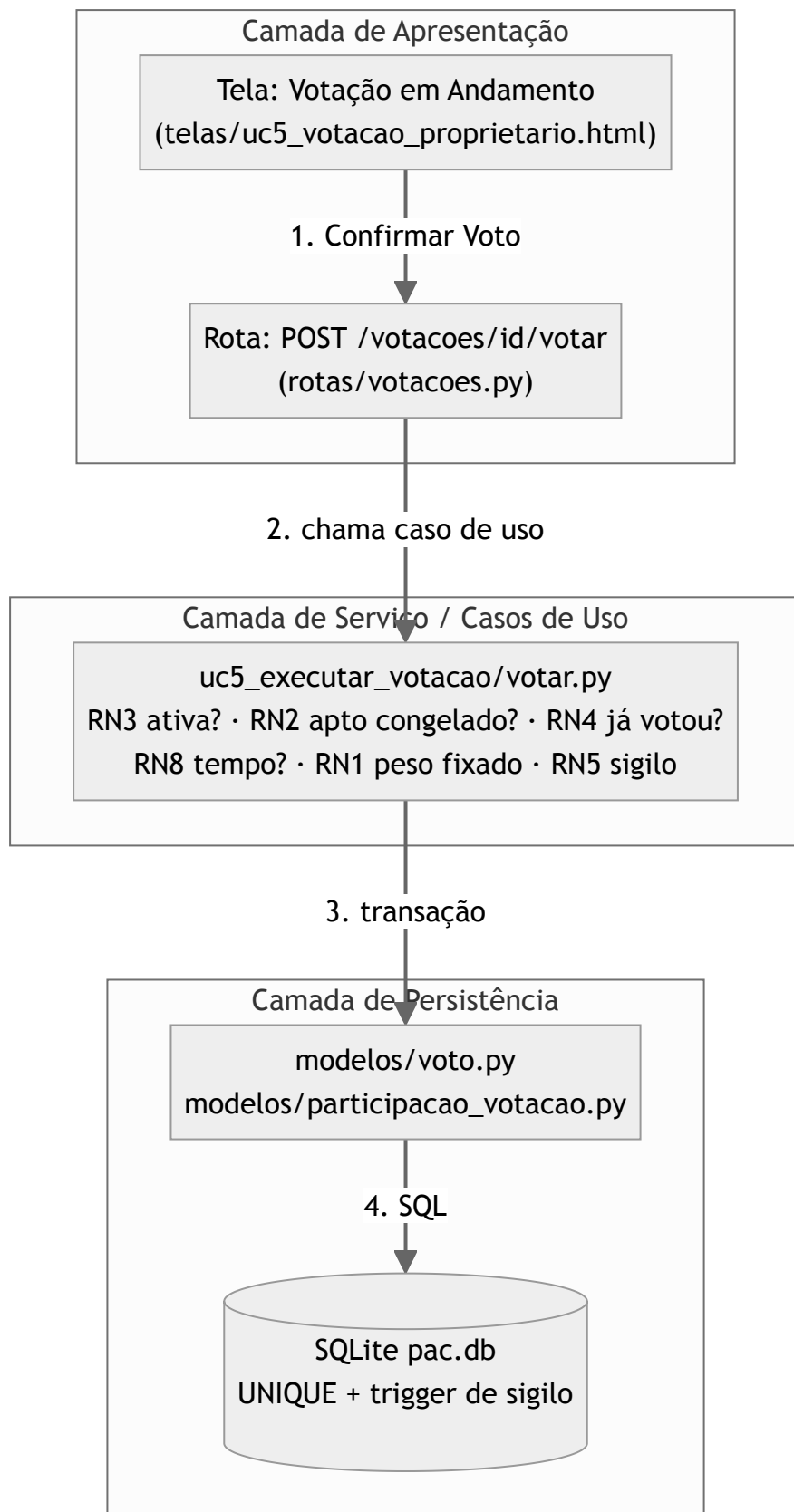
Alternativa descartada — microserviços: custos de infraestrutura e consistência distribuída sem benefício para o porte (~100 usuários, RNF3). Pior: RN1/RN2 exigem congelar aptos e pesos **transacionalmente** no Iniciar — trivial num monólito com banco único, complexo entre serviços.

3. Stack tecnológica

TECNOLOGIA	PAPEL	JUSTIFICATIVA
Python 3.10+	Linguagem	Legibilidade máxima: cada regra de negócio vira uma função curta e autoexplicativa.
Flask 3	HTTP e rotas	Microframework sem "mágica": rotas explícitas URL → caso de uso; já embute Jinja2 e sessão assinada. Única dependência do projeto.

SQLite (stdlib sqlite3)	Banco (RNF2)	Gratuito, arquivo único, zero instalação; constraints reais (UNIQUE, FK, CHECK) essenciais para RN4/RN5 no banco.
Sessão do Flask	Sessão (UC1)	Cookie assinado httpOnly, suficiente para o porte (RNF3).
Werkzeug scrypt	Hash de senha (RNF4)	Hash forte com salt (equivalente a bcrypt), já acompanha o Flask.
Jinja2 + HTML/CSS puro	Telas (RNF1)	1 template por protótipo da ERSw; mobile-first responsivo desde 360 px; JS mínimo (cronômetro e polling).

4. Diagrama de arquitetura – fluxo "Confirmar Voto"

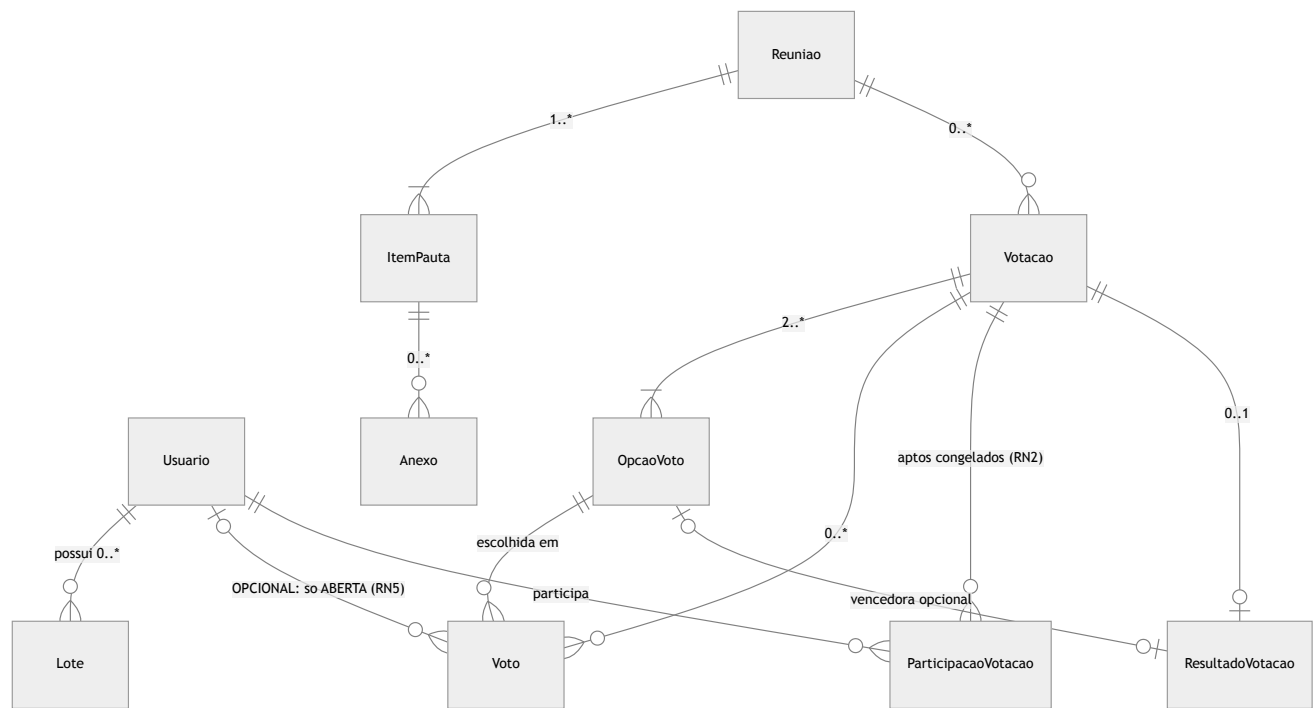


Retorno (passos 5–7): o banco responde ok ou violação de constraint (5); o caso de uso traduz em sucesso ou mensagem oficial da ERSw, ex.: *"Seu voto já foi registrado"* (6); a rota renderiza a tela com o resultado (7).

O mesmo desenho vale para todas as funcionalidades: **tela** → **rota** → **caso de uso** → **modelo** → **banco**, sempre nessa ordem, sem atalhos.

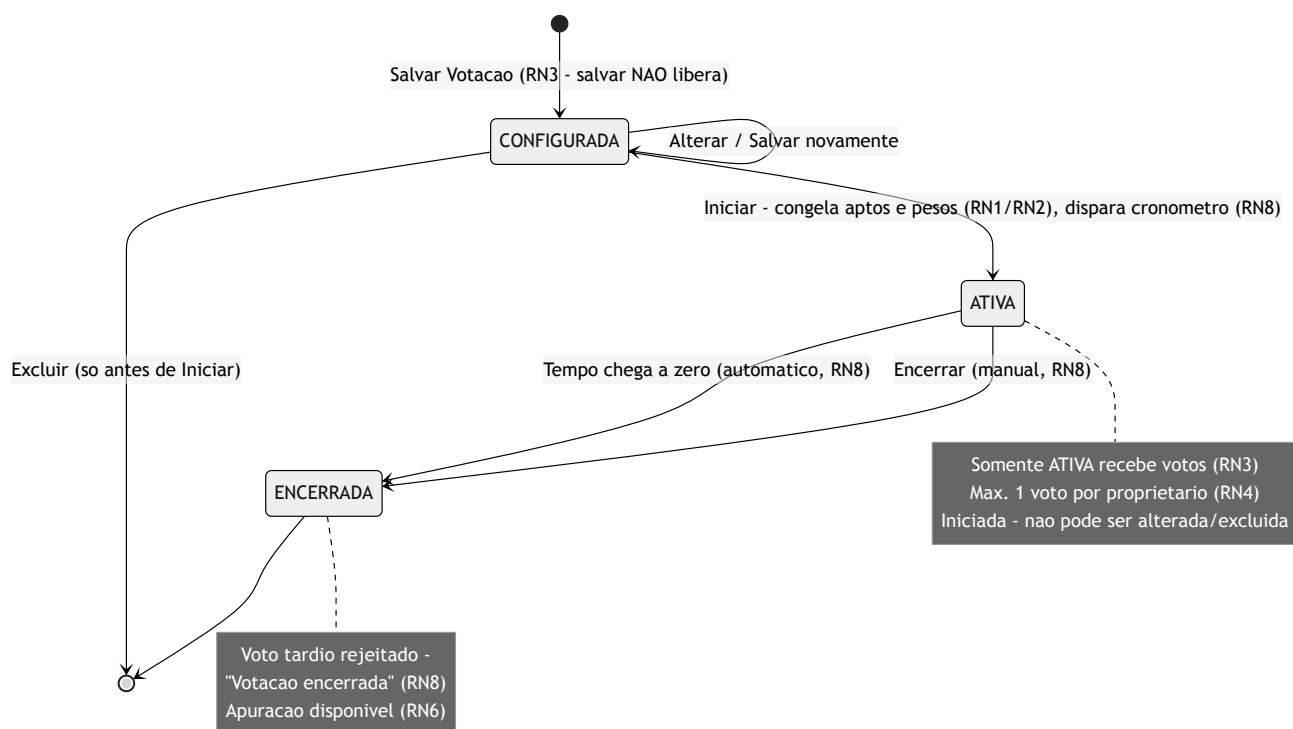
5. Diagrama de entidades

Dois pontos estruturais garantem o sigilo (RN5) **no próprio esquema**: `Voto → Usuario` é **opcional** (NULL em votação fechada) e `ParticipacaoVotacao` **nunca referencia** `OpcaoVoto`.



ENTIDADE	ATRIBUTOS PRINCIPAIS	OBSERVAÇÃO ESTRUTURAL
Usuario	nome, email, senha, telefone, tipo, ativo	Não possui peso próprio — o peso deriva dos lotes (RN1)
Lote	identificacao, fracaoIdeal, peso	Fonte oficial do peso de voto; pertence a exatamente 1 proprietário
Votacao	pergunta, tipoResposta, visibilidade, duracao (mm:ss), quorumMinimo, status, dataInicio, dataEncerramento	Status: CONFIGURADA → ATIVA → ENCERRADA (RN3)
OpcaoVoto	texto	2..* por votação (RN7)
Voto	dataHora, pesoComputado	usuario_id NULL em votação FECHADA (RN5) — trigger no banco garante
ParticipacaoVotacao	pesoFixado, dataHoraParticipacao	Nunca referencia OpcaoVoto (RN5); UNIQUE (votacao, usuario) reforça RN4
ResultadoVotacao	status, pesoTotalApto, pesoParticipante, percentualParticipacao, dataApuracao	opcaoVencedora NULL em EMPATE e SEM_QUORUM (RN6)

6. Máquina de estados da votação



7. Estrutura de pastas

```
trabalho-engenharia-1-/  
├── docs/ARQUITETURA.md          # este documento (fonte, com diagramas Mermaid)  
├── seed/popular_banco.py         # dados de teste – substitui UC2/UC3  
├── src/  
│   ├── config/                  # banco.py (esquema + constraints RN4/RN5), sessao.py,  
│   ├── constantes.py            # 1 arquivo por entidade persistente da ERSw (10 arquivos)  
│   ├── modelos/                 # * coração didático – regras de negócio comentadas  
│   ├── casos_de_uso/            └── uc1_autenticacao/entrar.py  
│   │   ├── uc4_criar_votacao/{criar,alterar,excluir}_votacao.py + validacoes.py  
│   │   ├── uc5_executar_votacao/{iniciar,votar,encerrar}.py  
│   │   └── uc6_resultado/{apurar,visualizar_resultado}.py  
│   ├── rotas/                   # URL → caso de uso (zero regra de negócio)  
│   ├── telas/                   # 1 template Jinja2 por protótipo da ERSw  
│   ├── publico/estilos.css      # CSS mobile-first (RNF1, desde 360 px)  
│   └── servidor.py              # ponto de entrada Flask  
└── README.md                    # instalação, credenciais e roteiro de demonstração
```

8. Tabela de rastreabilidade

FUNCIONALIDADE	UC / RF	RN	ARQUIVOS
Entrar no sistema	UC1 / RF9, RNF4	—	uc1_autenticacao/entrar.py · rotas/autenticacao.py · telas/login.html
Criar votação	UC4 / RF5	RN3, RN7,	uc4_criar_votacao/* · telas/uc4_configurar_votacao.html

		RN8, RN9	
Executar — Iniciar	UC5 / RF6	RN1, RN2, RN3, RN8	uc5_executar_votacao/iniciar.py · modelos/participacao_votacao.py
Executar — Votar	UC5 / RF6	RN1– RN5, RN8	uc5_executar_votacao/votar.py · config/banco.py (UNIQUE + trigger) · telas/uc5_votacao_proprietario.html
Executar — Encerrar	UC5 / RF6	RN8	uc5_executar_votacao/encerrar.py · telas/uc5_conducao_admin.html
Verificar resultado	UC6 / RF7	RN5, RN6	uc6_resultado/apurar.py , visualizar_resultado.py · telas/uc6_resultado_{aberta, fechada}.html
Dados de teste	seed (RF1– RF3)	RN1, RN2	seed/popular_banco.py

9. Decisões de segurança

- **Hash de senha (RNF4):** Werkzeug `generate_password_hash` (scrypt, salt por senha); login compara com `check_password_hash`.
- **Política de senha (RNF4):** mínimo 8 caracteres + rejeição de lista embutida de senhas comuns.
- **Mensagem não enumerável (UC1):** qualquer falha de login exibe sempre "*E-mail ou senha incorretos*", sem revelar qual campo errou.
- **Sessão:** cookie assinado httpOnly; rotas protegidas por perfil com decorators (`@exige_admin`, `@exige_proprietario`).
- **Sigilo da fechada no banco (RN5/RNF5):** em votação FECHADA o `Voto` é gravado com `usuario_id` NULL — a associação participante→escolha **não existe fisicamente**; um trigger rejeita qualquer tentativa de gravá-la; `ParticipacaoVotacao` não tem coluna de opção.
- **Voto único sob concorrência (RN4):** verificação + gravação na mesma transação; `UNIQUE` (`votacao_id`, `usuario_id`) faz o banco rejeitar duplicata mesmo em requisições simultâneas → traduzida para "*Seu voto já foi registrado*".

10. Divisão da apresentação — 3 membros

Cada bloco é coeso, tem fronteira de pastas (nenhum arquivo pertence a dois apresentadores) e um momento próprio na demonstração ao vivo. A atribuição de nomes abaixo é uma sugestão — os blocos são independentes e podem ser trocados entre os membros.

Membro 1 — Arthur Magno · Fundações e Entrada (UC1)

Conteúdo: visão geral e arquitetura em camadas (seções 1–7); modelo de dados e constraints RN4/RN5 no banco; seed; login (RF9/RNF4).

Arquivos para apontar: docs/ARQUITETURA.md · src/config/banco.py · src/modelos/ · seed/popular_banco.py · src/casos_de_uso/uc1_autenticacao/entrar.py · src/telas/login.html

Demonstra: seed + login válido (admin e proprietário) e inválido.

Membro 2 — João Gabriel · Votação: criação e execução (UC4 + UC5)

Conteúdo: criar votação (3 tipos, visibilidade, duração RN8, **Salvar ≠ Iniciar** RN3); executar (Iniciar congela aptos/pesos RN1/RN2, cronômetro, voto único RN4, fluxos de erro).

Arquivos para apontar: src/casos_de_uso/uc4_criar_votacao/ · src/casos_de_uso/uc5_executar_votacao/ · src/telas/uc4_configurar_votacao.html · uc5_conducao_admin.html · uc5_votacao_proprietario.html

Demonstra: criar e iniciar votação, Maria votando com peso 2,40, os 4 fluxos de erro.

Membro 3 — Maria Eduarda Gomes · Resultado, sigilo e encerramento (UC6)

Conteúdo: apuração RN6 (quórum, CONCLUIDO/EMPATE/SEM_QUORUM, abstenção não concorre); sigilo da votação fechada (RN5/RNF5) com prova no banco; encerramento manual/automático (RN8).

Arquivos para apontar: src/casos_de_uso/uc6_resultado/apurar.py · visualizar_resultado.py · uc5_executar_votacao/encerrar.py · src/telas/uc6_resultado_aberta.html · uc6_resultado_fechada.html

Demonstra: encerramento, resultado Aberta × Fechada, EMPATE e SEM_QUORUM.

11. Roteiro de demonstração ensaiado

Preparação (antes de apresentar): rodar `.venv/bin/python seed/popular_banco.py` e `.venv/bin/python src/servidor.py`; abrir **duas janelas** de navegador (normal = administradora; anônima = proprietários) em `http://127.0.0.1:5001`. Senha de todos: `Condominio@2026`. Sugestão de tempo total: 12–15 min (4–5 min por membro).

Bloco 1 — Arthur Magno (fundações + login)

- 1 "O PAC segue arquitetura em camadas: tela → rota → caso de uso → banco. Cada regra da ERSw mora num arquivo nomeado pelo caso de uso." — mostrar a árvore `src/casos_de_uso/` e o diagrama da seção 4.
- 2 Rodar o seed no terminal e ler a saída em voz alta: 4 aptos (Maria 2,40 pela soma dos 2 lotes — RN1), Pedro sem lote e Otávio inativo fora (RN2), peso total 6,30.
- 3 Tentar login com senha errada → "E-mail ou senha incorretos" — a mesma mensagem para e-mail inexistente ou usuário inativo: não revelamos qual campo errou (RNF4)."
- 4 Login `carla@condominio.com` → painel do administrador. Na janela anônima, login `maria@condominio.com` → painel da proprietária com "Peso dos seus lotes: 2,40". "Perfis diferentes, painéis diferentes — a sessão guarda o perfil e decorators protegem as rotas."
- 5 Fechar mostrando `banco.py`: "o sigilo e o voto único não dependem só do código: são UNIQUE e trigger no próprio banco." — deixa o gancho para os blocos 2 e 3.

Bloco 2 — João Gabriel (criar + executar)

- 1 Como Carla: **Nova Votação** → preencher "Aprova o orçamento anual?", tipo Sim/Não/Abstenção, Aberta, tempo `05:00`, quórum `30` → **Salvar Votação**. Status: Configurada. "Salvar não libera (RN3) — reparem que o botão Iniciar fica na listagem, fora do formulário."
- 2 Provocar validações (RN7/RN8): tentar salvar outra votação com tempo `00:00` → rejeitada; Múltipla Escolha com 1 opção → rejeitada, dados preservados.
- 3 Clicar **Iniciar** → tela de Condução: peso apto 6,30, cronômetro regressivo, votaram 0/4. "Neste clique o PAC fotografou os aptos e pesos em ParticipacaoVotacao — mudanças de lote depois disso não afetam esta votação (RN1/RN2)."
- 4 Na janela anônima (Maria): **Votar** → "Peso do seu voto: 2,40 (soma dos lotes Apt 202; Apt 203)" → confirmar **Sim** → "Voto confirmado com peso 2,40"; a Condução atualiza sozinha.
- 5 Fluxos de erro, um a um: Maria vota de novo → "Seu voto já foi registrado" (RN4); login `pedro@` e abrir a votação → "Você não está apto a votar nesta votação" (RN2); tentar **Iniciar** outra votação → bloqueado: já existe Ativa na reunião.
- 6 Ainda com a votação ativa, votar com `jaoa@` (Não) e `bruno@` (Abstenção) para preparar o bloco 3.

Bloco 3 — Maria Eduarda Gomes (encerrar + resultado + sigilo)

- 1 Como Carla, na Condução: **Encerrar**. Voltar à aba de um proprietário que não votou (Ana) e tentar votar → "Votação encerrada" — vale igual quando o cronômetro zera: o servidor encerra sozinho e rejeita voto atrasado (RN8)."

- 2 **Ver Resultado** (Aberta): quórum 76,2% $\geq$ 30% Atingido; CONCLUIDO — Aprovado (Sim 2,40 $\times$ Não 1,25); relação nominal com lotes, voto e peso. "A Abstenção do Bruno contou no quórum, mas não concorre (RN6)."
- 3 Criar + iniciar uma votação **Fechada** (Múltipla Escolha: Construtora Alfa/Beta), votar com 2–3 proprietários, encerrar → resultado: *apuração consolidada* + "Participantes da votação (sem escolha individual)".
- 4 A prova do sigilo no banco (RN5/RNF5), no terminal:

```
sqlite3 pac.db "SELECT usuario_id, peso_computado FROM voto WHERE votacao_id=<id>;"
```

usuario_id vem vazio (NULL) em todos — "a associação não existe fisicamente; nem uma consulta SQL direta recupera quem votou em quê. E um trigger impede gravá-la por engano."
- 5 **EMPATE:** nova votação Sim/Não sem quórum → Maria vota Sim (2,40); João (1,25) e Bruno (1,15) votam Não (2,40) → encerrar → EMPATE, vencedora vazia, "Empate entre: Sim · Não".
- 6 **SEM QUÓRUM:** nova votação com quórum 50 → só Maria vota (38,1%) → encerrar → SEM_QUORUM — Não atingido, sem vencedora. Fechamento: "os três status da RN6 demonstrados, com apuração sempre ponderada por peso."

Credenciais de teste

E-MAIL	PERFIL	LOTES	PESO
carla@condominio.com	Administradora	—	—
joao@condominio.com	Proprietário	Apt 101	1,25
maria@condominio.com	Proprietário	Apt 202 + Apt 203	2,40 (RN1)
ana@condominio.com	Proprietário	Casa 05	1,50
bruno@condominio.com	Proprietário	Apt 304	1,15
pedro@condominio.com	Proprietário	nenhum	não apto (RN2)
otavio@condominio.com	Proprietário inativo	Apt 401	não apto (RN2)

Senha de todos: `Condominio@2026`